

UNIT - IV

SYLLABUS

Distributed File System : Introduction to Distributed File System, Architecture and Mechanism for Building Distributed File System.

Distributed Shared Memory : General Architecture of DSM systems, Algorithm for Implementing DSM, Memory coherence and Coherence Protocols.

INTRODUCTION

A distributed file system is a client/server-based application that allows clients to access and process data stored on the server as if it were on their own computer. When a user accesses a file on the server, the server sends the user a copy of the file, which is cached on the user's computer while the data is being processed and is then returned to the server.

Ideally, a distributed file system organizes file and directory services of individual servers into a global directory in such a way that remote data access is not location-specific but is identical from any client. All files are accessible to all users of the global file system and organization is hierarchical and directory based.

Since more than one client may access the same data simultaneously, the server must have a mechanism in place (such as maintaining information about the times of access) to organize updates so that the client always receives the most current version of data and that data conflicts do not arise. Distributed file systems typically use file or database replication (distributing copies of data on multiple servers) to protect against data access failures.

INTRODUCTION TO DISTRIBUTED FILE SYSTEM

Q.1. What is a file system?

Ans. File system :

- File systems are abstractions that enable users to read, manipulate and organize data. Typically the data is stored in units known as files in a hierarchical tree where the nodes are known as directories.
- The file system enables a uniform view, independent of the underlying storage devices which can range between anything from floppy drives to hard drives and flash memory cards.

- Since file systems evolved from stand-alone computers the connection between the logical file system and the storage device was typically a one-to-one mapping. Even software RAID that is used to distribute the data on multiple storage devices is typically implemented below the file system layer.
- A file system is responsible for the organization, storage, retrieval, naming, sharing and protection of files.
- File systems provide directory services, which convert a file name (possibly a hierarchical one) into an internal identifier (i.e. inod FAT under). They contain a representation of the file data itself and methods for accessing it (read/write).
- The file system is responsible for controlling access to the data and for performing low-level operations such as buffering frequently used data and issuing disk I/O requests.
- A file system is basically persistent stored data sets. It consists of a hierarchical name space. The name space is visible to all processes in the systems. The main features to a file systems are :
 - (1) It consists of various application program interfaces. These interfaces must be able to access and update operations on persistently stored data sets. Mostly the API's have sequential access model but additional random facilities provided.
 - (2) The file system allow sharing of data between users with the access control.
 - (3) The file system allows concurrent access to file systems. If concurrent users are accessing a read only file then it is absolutely fine, but when the same file is updated by two users certain prevention measures like locking, serializability, time stamping etc. are taken to maintain consistent data.

Q.2. What are the characteristics of file systems?

Ans. Characteristics of file systems :

- Files are stored on disks or other storage media. It is the responsibility of the file system to organize, store, share, name, retrieve and protect the files.

(1) Organize and store :

- A file consists of data and attributes. The data items are read and written mostly in a sequential fashion.
- The attributes comprises of file length, creation timestamp, read timestamp, write timestamp, attribute timestamp, reference count, owner, file type, access control list, etc.
- Most of the attributes like file length, creation timestamp etc. are managed by the file systems. The users are not allowed to update the attributes.

(2) Naming :

- The file system supports naming of the files with the help of directories. The directory module maps the text names of files to internal file identifiers.
- The directories may include subdirectories and files.

(3) Control access :

- The file system takes care of control access. Unauthorized users are not allowed to access the files.
- File system operations are as follows :
 - Read a file.
 - Write the contents on to a file.
 - Update the file.
 - Copy a file.
 - Transfer a bytes of data from a file.
 - Open a file.
 - Close a file.
 - Move file pointer to required position.
 - Get the file attributes.

Q.3. What is distributed file system?

Ans. Distributed file systems :

- Distributed file systems (DFS) are a part of distributed systems. DFS do not directly serve to data processing. They allow users to store and share data.

- They also allow users to work with these data as simply as if the data were stored on the user's own computer.
- Compared to a traditional client-server solution, where the data are stored on one server, important or frequently required data in DFS can be stored on several nodes (node means a computer operating as a DFS). This is called replication.
- Replication can be used for achieving better system performance and/or for achieving reliability of the system.
- The data in a DFS are more protected from a node failure. If one or more nodes fail, other nodes are able to provide all functionality. This property is also known as availability or reliability.
- Availability means that the system can serve client a request at a moment when the client connects to the system. Reliability means that the system is available all the time when the client is connected to it.
- Files can also be moved among nodes. This is typically invoked by an administrator and it is done for improving a load-balancing among nodes.
- The users should be unaware of where the services are located and also the transferring from a local machine to a remote one should also be transparent. In DFS, this property is known as transparency.
- If the capacity of the nodes is not enough for storing files, new nodes can be added to the existing DFS to increase DFS capacity. This property is known as scalability.
- A client usually communicates with the DFS using LAN, which is not a secure environment. Clients must prove their identity, which can be done by authenticating themselves to an authentication entity in the system.
- The data which flow between the client and the node must be resistant against attackers. This property is known as security.
- Special features of distributed file system are as follows :
 - It provides file system services. These services include storage, access, structure, protection of files on multiple machines. This is done transparently to the same way as on a single machine.
 - The distributed file systems allow sharing of multiple heterogeneous local file systems.

फोटोकॉपी (झेराक्स) करने से मैटर बहुत छोटा हो जाता है और इसे पढ़ने से आपकी आँखें कमजोर होती हैं।

(3) It also gives us the basic distinction between file servers and file service.

- The file service depends on the specification and the interface, whereas the server is a process that implements parts of file service.

Q.4. What are the requirements of distributed file system?

Ans. Distributed file system requirements :

- Many of the requirements and potential pitfalls in the design of distributed services were first observed in the early development of distributed file system.

- Initially, they offered access transparency and location transparency, performance, scalability, concurrency control, fault tolerance and security requirements emerged and were met in subsequent phase of development.

(1) Transparency :

- The file service is usually the most heavily loaded service in an intranet, so its functionality and performance are critical. The design of the file service should support many of the transparency requirements for distributed systems.

- The design must balance the flexibility and scalability that derive from transparency against software complexity and performance.

- The following forms of transparency are partially or wholly addressed by current file service :

(i) **Access transparency :** Client programs should be unaware of the distribution of files. A single set of operations is provided for access to local and remote files. Programs written to operate on local files are able to access remote files without modification.

(ii) **Location transparency :** Client programs should see a uniform file name space. Files or groups of files may be relocated without changing their pathnames and user programs see the same name space wherever they are executed.

(iii) **Mobility transparency :** Neither client programs nor system administration tables in client nodes need to be changed when files are moved. This allows file mobility-files or, more commonly, sets or volumes of files may be moved, either by system administrators or automatically.

(iv) **Performance transparency :** Client programs should continue to perform satisfactorily while the load on the service varies within a specified range.

(v) **Scaling transparency :** The service can be expanded by incremental growth to deal with a wide range of loads and network sizes.

(2) Concurrent file updates :

- Changes to a file by one client should not interfere with the operation of other clients simultaneously accessing or changing the same file. This is the well-known issue of concurrency control.

- The need for concurrency control for access to shared data in many applications is widely accepted and techniques are known for its implementation, but they are costly.

(3) File replication :

- In a file service that supports replication, a file may be represented by several copies of its contents at different locations. This has two benefits-it enables multiple servers to share the load of providing a service to clients accessing the same set of files, enhancing the scalability of the service and it enhances fault tolerance by enabling clients to locate another server that holds a copy of the file when one has failed.

- Few file services support replication fully, but most support the caching of files or portions of files locally, a limited form of replication.

(4) Hardware and operating system heterogeneity:

The service interfaces should be defined so that client and server software can be implemented for different operating systems and computers. This requirement is an important aspect of openness.

(5) Fault tolerance :

- The central role of the file service in distributed systems makes it essential that the service continue to operate in the face of client and server failures.

- Fortunately, a moderately fault-tolerant design is straightforward for simple servers. To cope with transient communication failures, the design can be based on at most-once invocation semantics. Or it can use the simpler at-least-once semantics with a server protocol designed in terms of idempotent operations operations, ensuring that duplicated requests do not result in invalid updates to files.

- The servers can be stateless, so that they can be restarted and the service restored after a failure without any need to recover previous state.

- Tolerance of disconnection or server failures requires file replication, which is more difficult to achieve.
- (6) **Consistency :**
- Conventional file systems such as that provided in UNIX offer one-copy update semantics. This refers to a model for concurrent access to files in which the file contents seen by all of the processes accessing or updating a given file are those that they would see if only a single copy of the file contents existed.
 - When files are replicated or cached at different sites, there is an inevitable delay in the propagation of modifications made at one site to all of the other sites that hold copies and this may result in some deviation from one copy semantics.
- (7) **Security :**
- Virtually all file systems provide access control mechanisms based on the use of access control lists.
 - In distributed file systems, there is a need to authenticate client requests so that access control at the server is based on correct user identities and to protect the contents of request and reply message, with digital signatures and (optionally) encryption of secret data.
- (8) **Efficiency :**
- A distributed file service should offer facilities that are of at least the same power and generality as those found in conventional file systems and should achieve a comparable level of performance.

ARCHITECTURE OF DISTRIBUTED FILE SYSTEM

Q.5. Draw and explain architecture of distributed file system.

Ans. Architecture of distributed file system :

- Ideally, in a distributed file system, files can be stored at any machine (or computer) and the computation can be performed at any machine (i.e. the machines are peers).

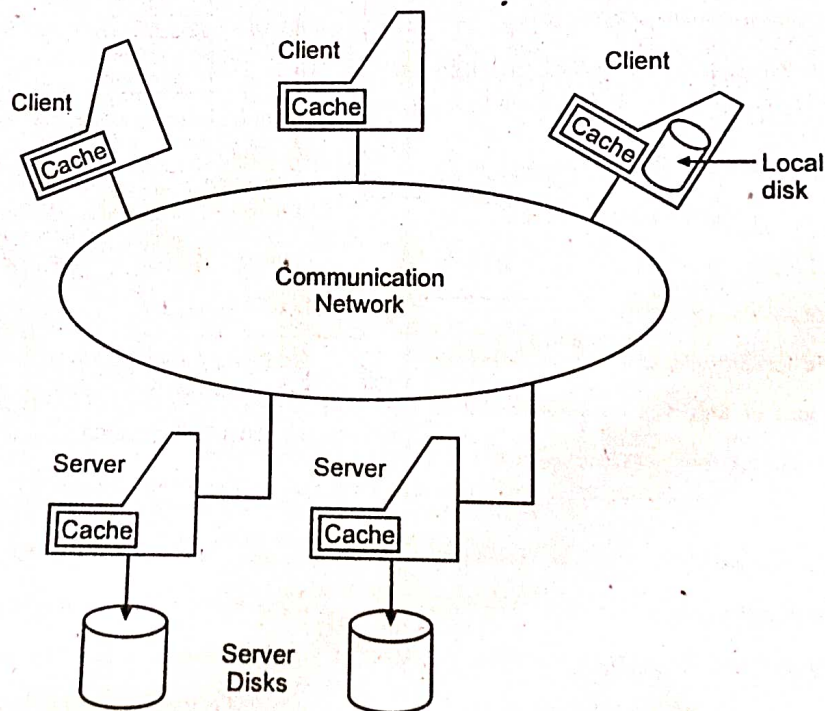


Fig. (a) Architecture of a distributed file system

- When a machine needs to access a file stored on a remote machine, the remote performs the necessary file access operations and returns data if a read operation is performed.
- However, for higher performance, several machines, referred to as file servers are dedicated to storing files and performing storage and retrieval operations.

- The rest of machines in the system can be used solely for communication purposes. These machines are referred to as client and they access the files stored on servers.
- Some client machines may also be equipped with a local disk storage that can be used for caching remote files, as a swap area or as a storage area.
- The two most important services present in a distributed file system are the name server and cache manager.
- A name server is a process that maps specified by clients to stored objects such as files and directories.
- The mapping (also referred to as name resolution) occurs when a process references a file or directory for the first time.
- A cache manager is a process that implements file caching. In file caching, a copy of data stored at a remote file server is brought to the client's machine when referenced by the client.
- Subsequent accesses to the data are performed locally at the client, thereby reducing the access delays due to network latency.
- Cache managers can be present at both clients and file servers.
- Cache managers at the servers caches files in the main memory to reduce delays due to disk latency.
- If multiple clients are allowed to cache a file and modify it, the copies become inconsistent.
- To avoid this inconsistency problem, cache managers at both servers and clients coordinate to perform data storage and retrieval operations. Typically, data access in a distributed file system is as shown in Fig. (b).

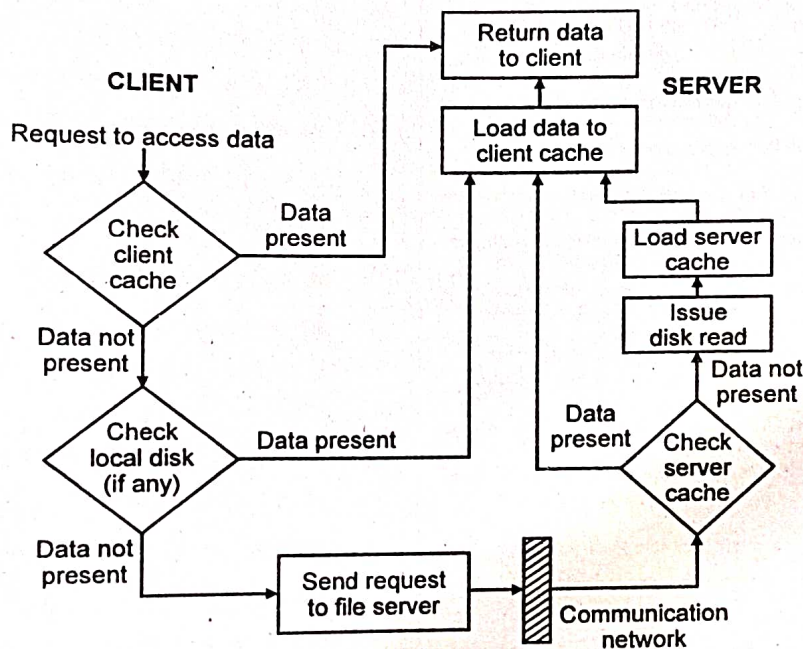


Fig. (b) Typical data access actions in distributed file system

- A request by a process to access a data block is presented to the local cache (client cache) of the machine (client) on which the process is running.
- If the block is not in the cache, then the local disk, if present is checked for the presence of data block.
- If the block is present, then the request is satisfied and the block is loaded into the client cache.
- If the block is not stored locally, then the request is passed on to the appropriate file server (as determined by the name server).
- The server checks its own cache for the presence of the data block before issuing a disk I/O request.
- The data block is transferred to the client cache in any case and loaded to the server cache if it was missing in the server cache.

MECHANISMS FOR BUILDING DISTRIBUTED FILE SYSTEMS

Q.6. Explain mounting mechanism for building distributed file system.

Ans. Mounting :

- A mount mechanism allows the binding together of different filename spaces to form a single hierarchically structured name

space. Even though mounting is UNIX specific, it is worthwhile to study this mechanism as most of the existing distributed file systems are based on UNIX.

- A name space (or a collection of files) can be bounded to or mounted at an internal node or a leaf node of a name space tree.
- Fig. illustrates a name space tree. A node onto which a name space is mounted is known as a mount point. In Fig. nodes a and i are mount points at which directories stored at server Y and server Z are mounted, respectively.
- Note that a and i are internal nodes in the name space tree. The kernel maintains a structure called the mount table, which maps mount points to appropriate storage devices.
- In the case of distributed file systems, file systems maintained by remote servers are mounted at the clients.
- There are two approaches to maintain the mount information :
 - (1) Mount information can be maintained at clients, in which case each client has to individually mount every required file system. This approach is employed in the Sun network file system. Since each client can mount a file system at any node in the name space tree, every client need not necessarily see an identical filename space.
 - (2) Mount information can be maintained at servers, in which case it is possible that every client sees an identical filename space. If files are moved to a different server, then mount information need only be updated at the servers. In the first approach, every client needs to update its mount table.

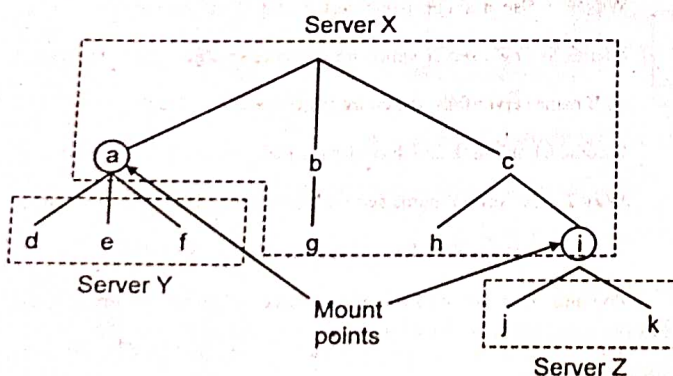


Fig. Name space hierarchy.

Q.7. Explain caching mechanism for building distributed file systems.

Ans. Caching :

- Caching is commonly employed in distributed file systems to reduce delays in the accessing of data.

- In file caching, a copy of data stored at a remote file server is brought to the client when referenced by the client.
- Subsequent access to the data is performed locally at the client, thereby reducing access delays due to network latency.
- Caching exploits the temporal locality of reference exhibited by programs. The temporal locality of reference refers to the fact that a file recently accessed is likely to be accessed again in the near future.
- Data can either be cached in the main memory or on the local disk of the clients. Also, data is cached in the main memory (server cache) at the servers to reduce disk access latency.
- These data include blocks swapped out at clients and data blocks that are contiguous to blocks previously requested by clients, as these data are more likely to be accessed soon.
- The file system performance can be improved by caching since accessing remote disks is much slower than accessing local memory or local disks.
- In addition, caching reduces the frequency of access to the file servers and the communication network, thereby improving the scalability of a file system.

Q.8. Explain hints mechanism for building distributed file system.

Ans. Hints :

- Caching improves file system performance by reducing the delay in accessing data. However, when multiple clients cache and modify shared data, the problem of cache consistency arises.
- Specifically, it must be guaranteed that the cached data is valid (up-to-date) and that a copy of the data recently updated in some other client cache or in the file server does not exist.
- Guaranteeing consistency is expensive in distributed file systems as it requires elaborate cooperation between file servers and clients.
- An alternative approach is to treat the cached data as hints. In this case, cached data are not expected to be completely accurate. However, valid cache entries improve performance substantially without incurring the cost of maintaining cache consistency.
- The class of applications that can utilize hints are those which can recover after discovering that the cached data are invalid.

For example, after the name of a file or a directory is mapped to the physical object, the address of the object can be stored as a hint in the cache. If the address fails to map to the object in the following attempt, the cached address is purged from the cache.

The file server consults the name server to determine the actual location of the file or directory and updates the cache.

Q.9. Explain bulk data transfer mechanism for building distributed file system.

Ans. Bulk data transfer :

The bulk of the delay in transferring data over a network is due to the high cost of executing communication protocols (such as the assembly and disassembly of packets, the copying of buffers between layers of the communication protocols, etc.)

In fact, actual transit time across a local area network can be insignificant. Transferring data in bulk reduces the protocol processing overhead at both servers and clients.

In bulk data transfer, multiple consecutive data blocks are transferred from servers to clients instead of just the block referenced by clients.

While caching amortizes the high cost of accessing remote servers over several local references to the same information, bulk transfer amortizes the protocol processing overhead and disk seek time over many consecutive blocks of a file.

Bulk transfers reduce file access overhead through obtaining a multiple number of blocks with a single seek; by formatting and transmitting a multiple number of large packets in a single context switch; and by reducing the number of acknowledgements that need to be sent. Bulk transfers exploit the fact that most files are accessed in their entirety.

Q.10. What are the design issues of distributed file system?

Ans. The design issues of distributed file system are as follows :

- (i) **Naming and name resolution :** Name refers to an object such as file or a directory. Name resolution refers to the process of mapping a name to an object that is physical storage. Name space is collection of names. Names can be assigned to files in distributed file system in three ways :

- (a) Concatenate the host name to the names of files that are stored on that host.

Advantages :

- (i) File name is unique.
- (ii) System wide.
- (iii) Name resolution is simple as file can be located easily.

Disadvantages :

- (i) It conflicts with the goal of network transparency.
- (ii) Moving a file from one host to another requires changes in filename and the application accessing that the file that is naming scheme is not location independent.
- (b) Mount remote directories onto local directories. Mounting a remote directory require that host of directory to be known only once. Once a remote directory is mounted, its files can be referred in location transparent way. This approach resolve file name without consulting any host.
- (c) Maintain a single global directory where all the files in the system belong to single namespace. The main limitation of this scheme is that it is limited to one computing facility or to a few cooperating computing facilities. This scheme is not used generally.

Name server : It is responsible for name resolution in distributed system. Generally two approaches are used for maintaining name resolution information.

Way 1 : Use a single name server that is all clients send their queries to single server which maps names to objects. Its limitation is: If name server fails, the entire system is affected and name server become a bottleneck and degrades the performance of the system.

Way 2 : Use several name servers (on different hosts) wherein each server is responsible for mapping objects stored in different domains. This approach is generally used. Whenever a name is to be mapped to an object, the local name server is queried. The local name server may point to remote server for further mapping of the name.

Example : "a/b/c" - requires a remote server mapping the b/c part of the filename. This procedure is repeated until the name is completely resolved.

- (2) **Caches on disk or main memory :** Caching refers to storage of data either into the main memory or onto disk space after its first reference by client machine.

Advantages :

- (i) Diskless workstations can also take advantage of caching.
- (ii) Accessing a cache in main memory is much faster than accessing a cache on local disk.
- (iii) The server cache is in the main memory at the server, a single design for a caching mechanism is used for clients and servers.

Disadvantages :

- (i) Large files cannot be cached completely so caching done block oriented which is more complex.
- (ii) It competes with virtual memory system for physical memory space, so a scheme to deal with memory contention cache and virtual memory system is necessary. Thus, more complex cache manager and memory management is required.

Advantages :

- (i) Large files can be cached without affecting performance.
 - (ii) Virtual memory management is simple.
 - (iii) Portable workstation can be incorporated in distributed system.
- (3) **Writing policy :** This policy decides when a modified cache block at client should be transferred to the server. Following policies are used :

- (a) **Write through :** All writes required by clients applications are also carried out at servers immediately. The main advantage is reliability that is when client crash, little information is lost. This scheme cannot take advantage of caching.
 - (b) **Delayed writing policy :** It delays the writing at the server that is modifications due to write are reflected at server after some delay. This scheme can take advantage of caching. The main limitation is less reliability that is when client crash, large amount of information can be lost.
 - (c) This scheme delays the updating of files at the server until the file is closed at the client. When average period for which files are open is short then this policy is equivalent to write through and when period is large then this policy is equivalent to delayed writing policy.
- (4) **Cache consistency :** When multiple clients want to modify or access the same data, then cache consistency problem arises. Two

schemes are used to guarantee that data returned to the client is valid

- (a) **Server initiated approach :** Server inform the cache manager whenever data in client caches becomes valid. Cache manager at clients then retrieve the new data or invalidate the blocks containing old data in cache. Server has maintain reliable records on what data blocks are cached by which cache managers. Co operation between servers and cache manager is required.

- (b) **Client-initiated approach :** It is the responsibility of cache manager at the clients to validate data with server before returning it to the clients. This approach does not take benefit of caching as the cache manager consult the server for validation of cached block each time.

- (5) **Availability :** It is one of the important issue in design of distributed file system. Server failure or communication network can attract the availability of files.

Replication : The primary mechanism used for enhancing availability of files is replication. In this mechanism, many copies or replicas of files are maintained at different servers.

Disadvantages :

- (i) Extra storage space is required to store replicas.
 - (ii) Extra overhead is required in maintained all replicas up to date.
- Following situations cause inconsistency among replicas :
- (i) Replica is not updated due to failure of server storing the replica.
 - (ii) All the file servers storing the replicas of file are not reachable from all clients due to network partition and replicas of file in different partition are updated differently.

Unit of replication : The main design issue in replication is the unit of replication. Following units can be used for replication of data :

- (a) **The most basic unit is file :** This unit allow the replication of only those files that need to have higher availability. This unit results in expansive replica management. Protection rights associated with directory have to individually stored with each replica. Replica of common directory may not have common file servers so require extra name resolution to locate each replica in case of modification of file or directory.
- (b) **Group of files called volumes can be used :** Volume may represent files of single user or files that are in a server. This scheme is used in

coda file system. In this scheme, replica management is simple that is protection rights can be associated with volume instead with each individual file replica. Volume replication results in wasteful as a user needs higher availability for only a few files in the volume.

- (c) **Combination of volume and single file replication can be used :** All the files of a user form a file group called primary pack. A replica of primary pack, called a pack, is allowed to contain a subset of files in primary pack. Corresponding to primary pack, one or more of packs can be obtained according to requirement.
- (6) **Scalability :** The design of DFS should be such that new systems can be easily introduced without affecting it. Generally, client server organization is used to define DFS structure. Caching is used in this organization to improve performance. Server initiated cached invalidation is used to maintain cache consistency. In this approach, server maintain a record based information regarding all the clients sharing file stored on it. This information represents server state. As the system grows both the size of server state and load due to invalidations increases on server. Following schemes can be used to reduce server state and server load :
- (a) Exploit knowledge about usage of files that is it is found that most commonly used and shared files are accessed in read only mode. So, there is no need to check the validity of these files of maintain the list of clients at servers of clients at servers for validation purpose.
- (b) Generally, data required by a client is found in another client's cache so a client can obtain required data from another client rather than server. Structure of server process play an important role. If server is designed with single process, then many clients have to wait for a long time whenever a disk input/output is initiated. This can be avoided if separate process is assigned to each client.
- (7) **Semantics :** The semantic of a file system represent the affects of accesses on file. The basic semantic is that a read operation will return the data (stored) due to latest write operation. The semantic can be guranteed in two ways: All read and writes from various clients will have to go through the server. Sharing will have to be disallowed either by server or by the use of locks by application. In first way, the server become bottleneck and in second way, the file is not available for certain clients.

DISTRIBUTED SHARED MEMORY

Q.11. What is distributed shared memory?

Ans. Distributed shared memory :

- Distributed shared memory system is a resource management component of a distributed operating system that implements the shared memory model in distributed system, which have no physically shared memory.
- The shared memory model provides a virtual address space that is shared among all nodes in a distributed system.
- Distributed shared memory (DSM) is an abstraction used for sharing data between computers that do not share physical memory. Processes access DSM by reads and updates to what appears to be ordinary memory within their address space.
- However, an underlying runtime system ensures transparently that processes executing at different computers observe the updates made by one another. It is as though the processes access a single shared memory, but in fact the physical memory is distributed.
- The main point of DSM is that it spares the programmer the concerns of message passing when writing applications that might otherwise have to use it. DSM primarily a tool for parallel applications or for any distributed application or group of applications in which individual shared data items can be accessed directly.
- DSM is in general less appropriate in client-server systems, where clients normally view server-held resources as abstract data and access them by request (for reasons of modularity and protection). However, servers can provide DSM that is shared between clients.
- For example, memory-mapped files that are shared and for which some degree of consistency is maintained are a from of DSM. (Mapped files were introduced with the MULTICS operating system.
- Message passing cannot be avoided altogether in a distributed system in the absence of physically shared memory, the DSM runtime support has to send updates in messages between computers.

- DSM systems manage replicated data, each computer has a local copy of recently accessed data items stored in DSM, for speed of access.

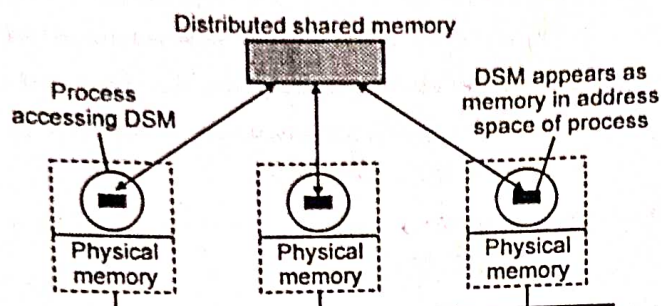


Fig. The distributed shared memory abstraction

GENERAL ARCHITECTURE OF DSM SYSTEMS

Q.12. Explain general architecture of distributed shared memory systems.

Ans. General architecture of distributed shared memory systems :

- With distributed shared memory, programs access data in the shared address space just as they access data in traditional virtual memory.
- In systems that support DSM, data moves between secondary memory and main memory as well as between main memories of different nodes.
- Each node can own data stored in the shared address space and the ownership can change when data moves from one node to another.
- When a process accesses data in the shared address space, a mapping manager maps the shared memory address to the physical memory (which can be local or remote).
- The mapping manager is a layer of software implemented either in the operating system kernel or as a runtime library routine.
- To reduce delays due to communication latency, DSM may move data at the shared memory address from a remote node to the node that is accessing data (when the shared memory addresses maps to a physical memory location on a remote node).
- In such cases, DSM makes use of the communication services of the underlying communication system.

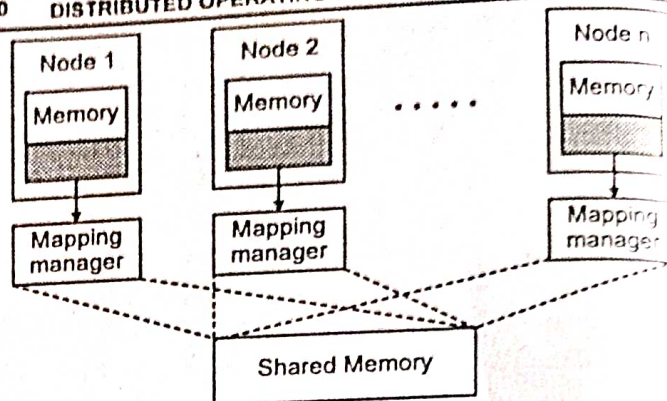


Fig. Distributed shared memory

Advantages of distributed shared memory :

- In the message passing model, programs make shared data available through explicit message passing. In other words, programmers need to be conscious of the data movement between processes. Programmers have to explicitly use communication primitives (such as SEND and RECEIVE), a task that places a significant burden on them. In contrast, DSM systems hide this explicit data movement and provide a simpler abstraction for sharing data that programmers are already well versed with. Hence, it is easier to design and write parallel algorithm using DSM rather than through explicit message passing.
- In the message passing model, data moves between two different address spaces. This makes it difficult to pass complex data structures between two processes. Moreover, passing data by reference and passing data structures containing pointers is generally difficult and expensive. In contrast, DSM systems allow complex structures to be passed by reference, thus simplifying the development of algorithm for distributed applications.
- By moving the entire block or page containing the data referenced to the site of reference instead of moving only the specific piece of data referenced, DSM takes advantage of the locality of reference exhibited by programs and thereby cuts down on the overhead of communicating over the network.
- DSM systems are cheaper to build than tightly coupled multiprocessor systems. This is because DSM systems can be built using off-the-shelf hardware and do not require complex interfaces to connect the shared memory to the processors.

- (5) The physical memory available at all the nodes of a DSM system combined together is enormous. This large memory can be used to efficiently run programs that require large memory without incurring disk latency due to swapping in traditional distributed systems. This fact is also favored by anticipated increases in processor speed relative to memory speed and the advent of very fast networks.
- (6) In tightly coupled multiprocessor system with a single shared memory, main memory is accessed via a common bus—a serialization point that limits the size of the multiprocessor system to a few tens of processors. DSM system do not suffer from this drawback and can easily be scaled upwards.
- (7) Program written for shared memory multiprocessors can in principle be run on DSM systems without any changes. At the least, such programs can easily be ported to DSM systems.
- In essence, DSM systems strive to overcome the architectural limitations of shared memory machines and to reduce the effort required to write parallel programs in distributed systems.

ALGORITHMS FOR IMPLEMENTING DSM

Q.13. Explain central-server algorithm of DSM.

Ans. The central-server algorithm :

- In the central-server algorithm, a central-server maintains all the shared data.
- It services the read request from other nodes or clients by returning the data items to them.
- It updates the data on write requests by clients and returns acknowledgement messages.
- A timeout can be employed to resend the requests in case of failed acknowledgements.
- Duplicate write requests can be detected by associating sequence numbers with write requests.
- A failure condition is returned to the application trying to access shared data after several retransmissions without response.
- While the central-server algorithm is simple to implement, the central-server can become a bottleneck.
- To overcome this problem, shared data can be distributed among several servers.

- In such a case, clients must be able to locate the appropriate server for every data access.
- Multicasting data access requests are undesirable as it does not reduce the load at the servers compared to the central-server scheme.
- A better way to distribute data is to partition the shared data by address and use a mapping function to locate the appropriate server.

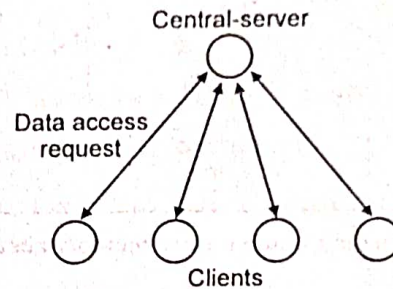


Fig. The central-server algorithm

Q.14. Explain migration algorithm of DSM.

Ans. The migration algorithm :

- In contrast to the central-server algorithm, where every data access request is forwarded to the location of data, data in the migration algorithm is shipped to the location of the data access request, allowing subsequent accesses to the data to be performed locally.
- The migration algorithm allows only one node to access a shared data a time.
- Typically, the whole page or block containing the data item migrates instead of an individual item requested.
- This algorithm takes advantage of the locality of reference exhibited by programs by amortizing the cost of migration over multiple accesses to the migrated between nodes while servicing only a few requests.
- To reduce thrashing, the Mirage system uses a tunable parameter that determines the duration for which a node possesses a shared data item.
- This allows a node to make a number of accesses to the page before it is migrated to another node.
- The Munin system strives to reduce data movement by employing protocols that are appropriate to different data access patterns.
- The migration algorithm provides an opportunity to integrate DSM with the virtual memory provided by the operating system running at individual nodes.

- When the page size used by DSM is a multiple of the virtual memory page size, a locally held shared memory page can be mapped to an application's virtual address space and accessed using normal machine instructions.

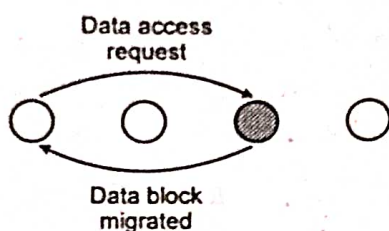


Fig. The migration algorithm

- On a memory access fault, if the memory address maps to a remote page, a fault-handler will migrate the page before mapping it to the process's address space.
- Upon migrating a page, the page is removed from all the address space it was mapped to the previous node.
- Note that several processes can share a page at a node.
- To locate a data block, the migration algorithm can make use of a server that keeps track of the location of pages or through hints maintained at nodes.
- These hints direct the search for a page towards the node currently holding the page. Alternatively a query can be broadcasted to locate a page.

Q.15. Explain read-replication algorithm of DSM.

Ans. The read-replication algorithm :

- In previous approaches, only processes on one node could access a shared data at any one moment.
- The read-replication algorithm extends the migration algorithm by replicating data blocks and allowing nodes to have read access or one node to have read-write access (the multiple readers - one writer protocol).
- Read-replication can improve system performance by allowing multiple nodes to access data concurrently.
- However, the write operation is expensive as all the copies of a shared block at various nodes will either have to be invalidated or updated with the current value to maintain the consistency of the shared data block.

- In read-replication algorithm, DSM must keep track of the location of all the copies of data blocks.
- Nevertheless, read replication has the potential to reduce the average cost of read operations when a ratio of reads to writes is large.

Q.16. Explain full-replication algorithm of DSM.

Ans. The full-replication algorithm:

- The full-replication algorithm is an extension of the read-replication algorithm.
- It allows multiple nodes to have both read and write access to shared data blocks (the multiple readers-multiple writes protocol).
- Because many nodes can write shared data concurrently, the access to shared data must be controlled to maintain its consistency.

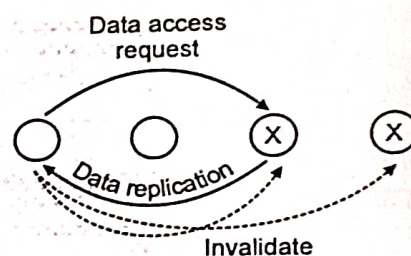


Fig. (a) Write operation in the read-replication algorithm

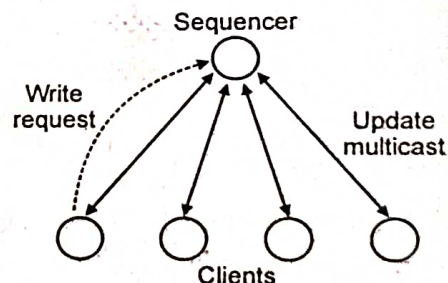


Fig. (b) Write operation in the full-replication algorithm

- One simple way to maintain consistency is to use gap-free sequencer.
- In this scheme, all nodes wishing to modify shared data will send the modifications to a sequencer.
- The sequencer will assign a sequence number and multicast the modification with the sequence number to all the nodes that have a copy of the shared data item.
- Each node processes the modification requests in the sequence number order.

- A gap between the sequence number of a modification request and the expected sequence number at a node indicates that one or more modifications have been missed.
- Under such circumstances, the node will ask for the retransmission of the modifications it has missed. (This implies that a log of the modifications is kept at some node).

MEMORY COHERENCE

Q.17. What is memory coherence?

OR What are several forms of memory coherence?

Ans. Memory coherence :

- To improve performance, DSM systems rely on replicating shared data items and allowing concurrent access at many nodes.
- However, if the concurrent accesses are not carefully controlled, memory accesses may be executed in an order different from that which the programmer expected.
- Informally, a memory is coherent if the value returned by a read operation is always the value that the programmer expected.
- For example, it is quite natural for programmer to expect a read operation to return a value stored by the most recent write operation. Thus, to maintain the coherence of shared data items, a mechanism that controls or synchronize the access is necessary. Also, to write correct programs, a programmer needs to understand how the concurrent updates to shared memory are carried out. The set of allowable memory access orderings forms the memory consistency model. The word consistency is used to refer to a specific kind of coherence. The most intuitive semantics for memory coherence is strict consistency, defined as follows; a read returns the most recently written value.
- Strict consistency requires a ability to determine the latest write, which in turn implies a total ordering of requests.
- The total ordering of request leads to inefficiency due to more data movement and synchronization requirements than what a program may really call for.
- To counter this problem, some DSM system attempts to improve the performance by providing relaxed coherence semantics.
- Following are several forms of memory coherence :

(1) Sequential consistency :

A system is sequentially consistent if the result of any execution of the operations of all the processors is the same as if they were executed in a sequential order and the operations of each individual processor appear in this sequence in this order specified by its program.

(2) General consistency :

A system supports general consistency if all the copies of a memory location eventually contain the same data when all the writes issued by every processor have completed.

(3) Processor consistency :

- Writes issued by a processor are observed in the same order in which they were issued. However, the order in which writes from two processors occur, as observed by themselves or a third processor, need not be identical.
- That is, two simultaneous reads of the same location from different processors may yield different results.

(4) Weak consistency :

- Synchronization accesses (accesses required to perform synchronization operations) are sequentially consistent. Before a synchronization access can be performed, all previous regular data accesses must be completed.
- Before a regular data access can be performed, all previous synchronization accesses must be completed. This essentially leaves the problem of consistency up to the programmer.
- The memory will only be consistent immediately after a synchronization operation.

(5) Release consistency :

- Release consistency is essentially the same as weak consistency, but synchronization accesses must only be processor consistent with respect to each other.
- Synchronization operations are broken down into acquire and release operations. All pending acquires (e.g. lock operation) must be done before a regular access is done and all regular accesses must be done after a release (e.g. an unlock operation) is done.
- Local dependencies within the same processor must still be respected.

- Release consistency is a further relaxation of weak consistency without a significant loss of coherence. In fact, the release consistency is differentiated from weak consistency by the inclusion of the following ordering relaxations.
- (i) Regular data accesses need not wait for release operations to complete, since release operations signal completion of regular accesses and are not concerned with the ordering of accesses following them.
- (ii) Acquire operations need not wait for the previous regular accesses to complete.
- (iii) Synchronization operations are only required to be processor consistent, not sequential consistent.

COHERENCE PROTOCOLS

Q.18. What are coherence protocols? Explain any one of them.

OR Explain in brief write-invalidate protocol.

Ans. Coherence protocols :

- To provide concurrent access, DSM systems make use of data replication, where copies of data are maintained at all the nodes accessing the data.
- A fundamental problem with data replication is the difficulty in ensuring that all copies have the same information and that nodes do not access stale data.
- In other words, a protocol to keep replicas coherent is needed.
- Two basic protocols to maintain coherence are the write-invalidate protocol and the write-update protocol.
- (1) **Write-invalidate protocol :**
 - In the write-invalidate method, a write to shared data causes the invalidation of all copies except one before the write can proceed.
 - Once invalidated, copies are no longer accessible.
 - A major disadvantage of this scheme is that invalidations are sent to all the nodes that have copies, irrespective of whether they will use this data or not.
 - This protocol is better suited for applications where several updates occur between read, as well as when a program exhibits a high degree of per-node locality of reference.
 - On the other hand, this protocol is inefficient if many nodes

frequently access an object, because an updated object will have to be copied back to many nodes immediately after every invalidation.

- The write-invalidate protocol has been used by a majority of DSM systems, which supports strict consistency.

(2) **Write-update protocol :**

- In the write-update method, a write to a shared data causes all copies of that data to be updated.
- This approach is more difficult to implement than the previous approach, as new value has to be sent instead of invalidation messages.
- This protocol can be expected to generate considerable network traffic.

Q.19. Explain cache coherence in the PLUS system.

Ans. Cache coherence in the PLUS system :

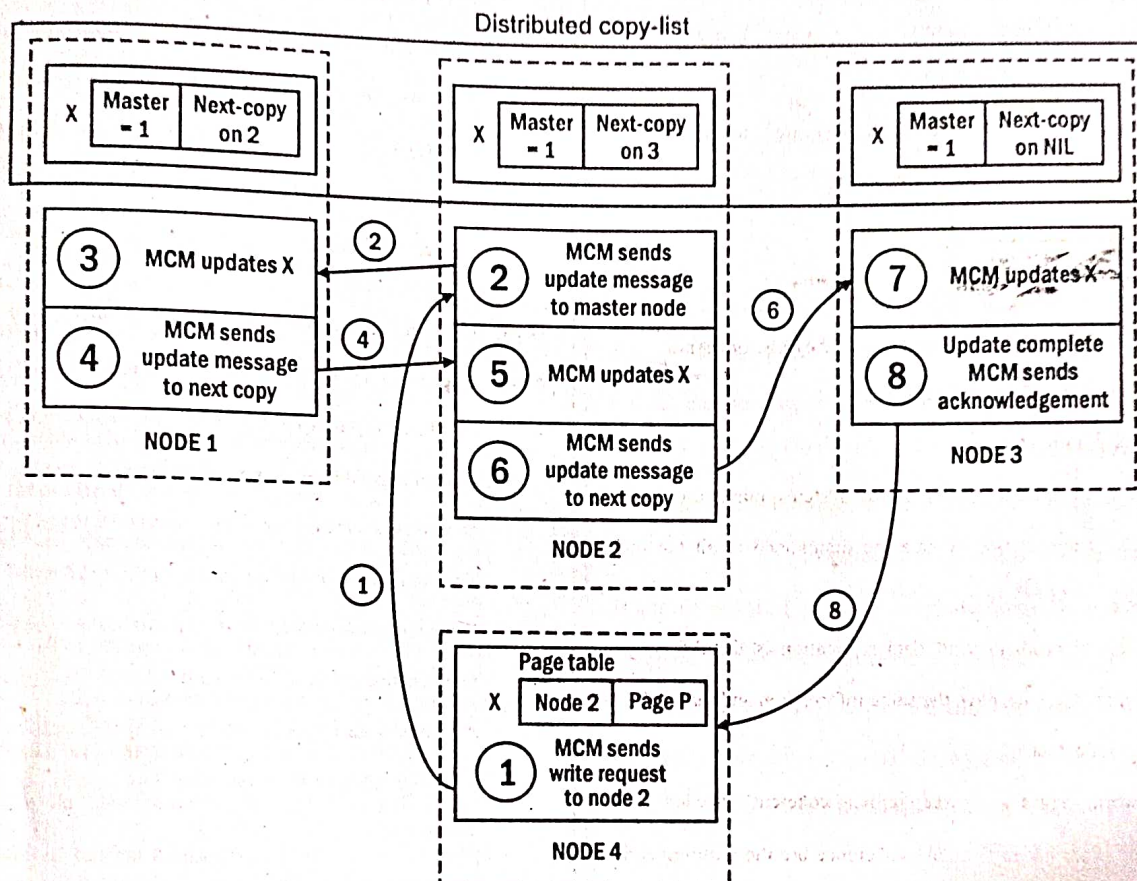
- The PLUS system employs the write-update protocol and supports general consistency. A memory coherence manager (MCM) running at each node is responsible for maintaining the consistency.
- The unit of replication is a page (4 Kbytes in the current implementation), however, the unit of memory access and coherence maintenance is one (32 bit) word.
- A virtual page in the PLUS system corresponds to a list of replicas of a page. One of the replicas is designated to be the master copy. The MCM on each node is made aware of the other replicas of a page through a distributed linked list called copy list (see Fig.)
- The copy-list is constructed by the operating system kernel and it has two pointers at each node, the master pointer and the next copy pointer.
- The master pointer points to the node storing the master copy, and the next-copy pointer points to a node containing another replica (if any) of the page along the copy-list.

Read operations :

- On a read fault, if the address indicates local memory, the local memory is read.
- Otherwise, the local MCM sends a read request to its counterpart at the specified remote node. The data returned by the remote MCM is passed back to the requesting processor.

Write operation :

- To ensure general consistency, writes are always performed first on the master copy and are then propagated to the copies linked by the copy-list.
 - On a write fault, if the local copy is not the master copy, then the update request is sent to the node containing the master copy for updating and then for further propagation.
 - On a write fault, if the address indicates a remote node (i.e. no local copy present), the update request is sent to the remote node.
 - In Fig. a write request is sent from node 4 to node 2.
 - If the copy on that node is not the master copy, then the update request is sent to the node containing the master copy for updating and then for further propagation.
- (In Fig. node 2, which is not the master of X, forwards the write request to node 1). To completely trace the steps of the write update protocol, see Fig.

**Fig. PLUS write-update protocol**

- The processor issuing the write operation is not blocked while the update operation is performed. However, if it initiates a read operation to the location currently being updated, it is blocked until the write completes. This is achieved by remembering the locations with pending writes.
- The above protocol guarantees strong ordering within a single processor independent of replication (in the absence of concurrent writes by other processors), but not with respect to another processor.
- When strong ordering is necessary for the correct synchronization between processors, write-fence operations must be explicitly used, wherein MCM waits for all previous writes to complete before performing subsequent writes.

Q.20. Explain unifying synchronization and data transfer in clouds.

Ans. Unifying synchronization and data transfer in clouds :

- Memory coherence and process synchronization are closely intertwined. Reading and writing of shared data by processes is

invariably controlled by a synchronization method.

- For example, in DSM systems using data replication and write invalidate protocol, the reader-writer problem can be implemented using locks as shown in Fig.

(*Writer process*)	(*Reader process*)
Loop	Loop
wait (empty);	wait (full);
writelock (buffer);	readlock (buffer);
update buffer;	read buffer;
unlock (buffer);	unlock (buffer);
signal (full);	signal (empty);
Endloop;	Endloop;

Fig. The reader-writer problem

- Since the lock operations are separated from the data transfer operations (update buffer and read buffer), separate messages must be sent to the node which currently owns the data for lock and data access operations. In addition, for a write operation, several invalidation messages must be sent.
- The sending of extra messages can be avoided by sending the data segment to the process whenever a process performs a lock operation on a data segment.
- Also, whenever a lock is released, the corresponding data segment is also returned to the owner node of the segment.
- In the reader-writer problem, when a reader performs an unlock operation, the buffer segment is returned to the owner. Therefore, when the last reader performs the unlock operation no reader will have a copy of the buffer segment. Now a writer process can update the buffer without sending any invalidation messages.

Q.21. Explain type-specific memory coherence in Munin system.

Ans. Type-specific memory coherence in Munin system:

- The cost of maintaining coherence can be reduced by exploiting application specific semantics information. Programmers can aid the system by providing semantics hints about the anticipated access pattern for a program's shared data objects.
- Making use of these hints, a DSM system can employ different coherence mechanisms, each appropriate for a different class of shared data objects. In the Munin system a shared data object is classified as one of nine classes based on the access pattern.
- On an access fault, an appropriate fault-handler specific to that object type is invoked to service the fault. Following are the type-specific coherence mechanisms in Munin :

Write-once objects :

Write-once objects are written during initialization, but only read afterwards. These objects are replicated on demand and are accessed locally at each site. For large objects, only selected portions of the object are replicated to avoid inefficient memory utilization.

Private objects :

Private objects are accessed by single threads even though they are accessible to all threads of a process. These objects are not managed by the memory coherence system. Should there be an attempt to access a private object from a remote thread, the object is brought under the purview of the coherence system.

Write-many objects :

- Write-many objects are frequently modified by multiple threads between synchronization points. To efficiently support write many-objects, Munin employs delayed updates.
- Whenever a node updates a replicated data object, the updates are buffered. Only when the thread synchronizes are updates propagated. (Note that the shared memory in Munin is weakly consistent).
- By delaying updates, Munin allows updates to the same object to be combined, therefore reducing data movement.

Result objects : Result objects are a restricted subset of write-many objects. These objects are not read until all parts of them are updated. This means that concurrent updates to different parts of a result object will conflict. Munin efficiently maintains, these objects through the delayed update mechanism.

Synchronization objects :

- Synchronization objects such as distributed locks can be employed to give threads exclusive access to data objects. Other synchronization objects, such as monitors are built on top of the distributed locks.
- In Munin, proxy objects, one per processor are used to implement distributed locks. A proxy object represents a remote object to all the threads in the local address space.
- A lock server at each processor maintains the local proxy object. A lock operation by a thread on the local proxy object causes the local

lock server to interact with the other lock servers to acquire the global lock associated with the local proxy.

Unlock operation is handled similarly. Processors exchange lock ownership in Munin. A queue associated with the lock contains a list of servers waiting to acquire the lock, thus facilitating ownership exchange.

Migratory objects :

Migratory objects are accessed in phase, where each phase corresponds to a series of accesses by a single thread. Objects accessed in a critical section fall under this class.

This class of objects is efficiently handled by combining lock requests with data movement, as done in the clouds system. That is, objects migrate to the site requesting the lock.

Producer-consumer objects :

Producer-consumer objects are typically written (produced) by one thread and read (consumed) by a fixed set of other threads. These types of objects are common in scientific programming.

Munin handles a producer-consumer type of object by moving the object to the site where it will be accessed in advance. This is referred to as eager object movement.

Read-mostly objects : Read mostly objects are read far more frequently than they are written. Munin provides efficient access to these class of objects by updating them through broadcasts. Broadcasts do not cause significant overhead as writes are infrequent.

General read-write objects :

General read-write objects do not exhibit particular pattern of access behaviour. However, this class of objects is rarely encountered.

For general read-write objects, Munin makes use of the Berkeley ownership protocol, which supports strict consistency. Under this protocol, an object at a node can be in one of the following states :

- (i) **Invalid :** The object does not contain useful data.
- (ii) **Unowned :** The object contains valid data. Other nodes may have copies of the object. The object cannot be updated without first acquiring ownership.
- (iii) **Owned exclusively :** The object contains valid data and is unique. The object can be updated locally. Upon request, the object must be shared with other nodes.

(iv) **Owned nonexclusively :** The object contains valid data, but it cannot be updated before invalidating other copies.

- Two types of read operations, read-shared and read-for-ownership, are provided in the Berkeley protocol. The objects not found locally are located through a broadcast query.
- On a read-shared fault, the owner node provides a copy of the object to the requesting node and changes the state of the object to "owned nonexclusively". The state of the copy of the object at the receiving node is unowned.
- On a read-for-ownership fault, the owner responds by sending a copy of the object and by marking its own copy as invalid. The node that receives a copy of the object becomes the new owner and marks the object as "owned exclusively."
- On a write operation, if the object is available locally and is in the owned exclusively state, the object can be updated without sending out any invalidating messages. On the other hand, if the object's state is "owned nonexclusively," then invalidation messages are sent to other nodes having copies of the object as invalid.
- On a write fault, if the object is not available locally or if the object is available locally, but its state is unowned, then the object's ownership must be acquired first by issuing a read-for-ownership operation before proceeding with the update.
- Currently, in the Munin system, the programmer provides all the semantic information. Once specified, this information remains fixed.

Q.22. Write a short note on Granularity.

Ans. **Granularity :**

- Granularity refers to the size of the shared memory unit. A page size that is a multiple of the size provided by the underlying hardware or the memory management system allows for the integration of DSM and the memory management systems.
- By integrating DSM with the underlying memory management system, a DSM system can take advantage of the built-in protection mechanism to detect incoherent memory references and use built-in fault handlers to prevent and recover from inappropriate references.

- A large page size for the shared memory unit will take advantage of the locality of reference exhibited by processes.
- By transferring large pages, less overhead is incurred due to paging activity and processing communication protocols.
- However, the larger the page sizes, the greater the chance for contention to access a page by many processes. Smaller page sizes are less apt to cause contention as they reduce the likelihood of false sharing.
- False sharing of a page occurs when two different data items, not shared but accessed by two different processes, are allocated to a single page.
- In the PLUS system, while the unit of replication is a page (4 Kbytes), the unit of memory access and coherence maintenance is one word (32 bits).
- This approach has the advantages of both the smaller and the bigger granularity.
- In Clouds and the Munin system, the unit of shared data structure is the object itself. Hence, the granularity size varies with the size of the object and false sharing does not occur in these systems.
- However designers of the Munin system study several parallel programs and show that coherence protocols move much larger units of memory than the program requires, if coherence is maintained on a per object basis.
- Thus, protocols that adapt to granularity size that is appropriate to the sharing pattern will perform better than those protocols that make use of a static granularity size.

Q.23. Write a short note on page replacement.

Ans. Page replacement :

- A memory management system has to address the issue of page replacement because the size of physical memory is limited.
- In DSM systems that support data movement, traditional method, such as least recently used (LRU) cannot be used directly.
- Data may be accessed in different modes such as shared, private, read-only, writable etc, in DSM systems.
- To avoid a degradation in the system performance, a page replacement policy would have to take the page access modes into consideration.
- For instance, private pages may be replaced before shared pages, as shared pages would have to be moved over the network, possibly to their owner.
- Read-only pages can simply be deleted as their owners will have a copy. Thus the LRU policy with classes is one possible strategy to handle page replacement.
- Once a page is selected for replacement, the DSM system must ensure that the page is not lost forever. One option is to swap the page onto disk memory. However, if the page is a replica and is not owned by the node, it can be sent to the owner node.
- Both the Methers system and the Memnet system make use of reserved memory, wherein each node is responsible for certain portions of the global virtual space and reserves memory space for those portions. A page is sent back to its reserved memory when selected for replacement elsewhere in the system.

POINTS TO REMEMBER :

- (1) A distributed file system is a component of a distributed operating system.
- (2) In a distributed file system, files can be stored at any machine (or computer) and the computation can be performed at any machine (i.e. the machines are peers).
- (3) Distributed shared memory system is a resource management component of a distributed operating system that implements the shared memory model in distributed system, which have no physically shared memory.
- (4) Maintaining copies of data coherent is a fundamental problem and a costly overhead associated with DSM systems.
- (5) In addition to maintaining the coherence of copies of replicated data, there is a need for the synchronization of accesses to shared data.